



AMERICAN INTERNATIONAL UNIVERSITY–BANGLADESH

Faculty of Science & Technology (FST)
Department of Computer Science

OBJECT ORIENTED PROGRAMMING 2

FINAL TERM PROJECT

GoMart Application

Retail Billing & Inventory Management System

Group Name	Team GoMart
Section	AA
Semester	Summer 2025-2026
Submitted To	Dr. Md Iftekharul Mobin
Student Name	Maisha Jannat Monzur
Student ID	22-47624-2
Email	maishajtmonzur@gmail.com
Group Member	Nawrin Parvin Adita (24-56038-1)
Contribution	50% of the group implementation

Individual Project Submission & Contribution Report

Table of Contents

1. Student & Project Information
 2. Project Overview
 3. GitHub & Repository Documentation
 4. Individual Code Contribution
 5. Database Contribution
 6. GUI / Windows Forms Evidence
 7. System Architecture & Diagrams
 8. Testing & Run Verification
 9. Academic Integrity & AI Disclosure
 10. Declaration
- Appendix — Submission Checklist

1. Student & Project Information

Student	Maisha Jannat Monzur
Student ID	22-47624-2
Email	maishajtmonzur@gmail.com
Section	AA
Course	Object Oriented Programming 2
Project	GoMart Application
Domain	Retail Billing & Inventory Management System
Group Name	Team GoMart
Group Members	Maisha Jannat Monzur (22-47624-2); Nawrin Parvin Adita (24-56038-1)
Number of Members	2
Individual Contribution	50%

1.1 Contribution Split

Maisha Jannat Monzur	50%	Login, Main Menu, Category, Product
Nawrin Parvin Adita	50%	Seller, Admin, Selling/Billing, About

Shared work: database integration, DBConnect.cs, project configuration, SQL/database resources, testing, integration of the modules, documentation and final repository organization were treated as shared group work.

2. Project Overview

GoMart Application is a Windows desktop-based retail billing and inventory management system developed for a small retail-store environment. It uses C# Windows Forms for the user interface, ADO.NET for database communication, and Microsoft SQL Server for persistent storage.

The application supports two roles, Admin and Seller. Administrators can manage categories, products, users and selling functions, while sellers are focused on the selling/billing workflow.

2.1 Main Objectives

- Provide role-based login for Admin and Seller users.
- Provide category and product CRUD functionality.
- Provide seller and administrator management.
- Provide a billing interface with order-total calculation.
- Store billing records in Microsoft SQL Server.
- Use a straightforward relational database structure.
- Provide a practical Windows Forms interface for a small retail store.

2.2 Scope

- Single-store, single-database desktop application.
- Admin and Seller roles.
- Category-to-product management.
- Cash-style billing and bill history.
- Online store, barcode scanning, multi-currency, payment gateway, tax/discount engine and advanced dashboards are outside the current scope.

3. GitHub & Repository Documentation

The project is maintained as a public GitHub repository. The repository contains the application package, README, database resources, SQL setup script, diagrams and application screenshots.

README	README.md at repository root
Application	GoMartApplication.zip
Database	Database/GoMartDatabase.bacpac
SQL setup	Database/SQL_DB/GoMartDB_SQLServer_Setup.sql
ER diagram	Documentation/Diagrams/ER_Diagram.png
Architecture	Documentation/Diagrams/System_Architecture.png
Login flow	Documentation/Diagrams/Login_Role_Flow.png
GUI screenshots	Documentation/Screenshots/
Individual report	Documentation/Report/

3.1 GitHub Evidence Fields

The project contribution percentage is set at 50% for each member as agreed for this submission. GitHub commit count, dates and commit URLs should be copied from the live repository history rather than guessed.

GitHub username	https://github.com/maishajtmonzur-08sss
Contribution	50%
Commit count	14
Commit URL(s)	https://github.com/maishajtmonzur-08sss/GoMart/commits/main/
Group-agreed role	50% implementation contribution

4. Individual Code Contribution

Maisha Jannat Monzur contributed approximately one half of the implementation work. The split was organized by functional modules so that each member had clear ownership while shared integration remained a joint responsibility.

Form1.cs	Login & role-based authentication
frmMain.cs	Main/home menu and role-based navigation
frmCategory.cs	Category management and CRUD
AddProduct.cs	Product management, validation, search and category filtering

4.1 Code Distribution

The distribution below is intended to show the functional ownership of the application code. Designer and resource files associated with each form follow the same ownership as the main .cs file.

- Maisha — Form1.cs: login, role selection and authentication flow.
- Maisha — frmMain.cs: home screen and role-based menu navigation.
- Maisha — frmCategory.cs: category CRUD and duplicate-category handling.
- Maisha — AddProduct.cs: product CRUD, validation, category selection and search/filtering.
- Nawrin — frmAddNewSeller.cs: seller management.
- Nawrin — AddAdmin.cs: administrator management.
- Nawrin — SellingForm.cs: point-of-sale billing and bill saving.
- Nawrin — AboutBox1.cs: About information.
- Shared — DBConnect.cs, Program.cs, App.config, SQL/database files, project integration and testing.

4.2 Non-Trivial Implementation Explanation

My main implementation responsibility in GoMart was the front part of the application where users enter the system and where administrators manage the product catalogue. I worked on the login form and the role-based navigation flow so that the selected Admin or Seller role is checked against the appropriate database table before the main form is opened. I also handled the category and product management forms. The category module supports adding, updating, deleting and listing categories, while the product module connects products with categories, validates price and quantity input, checks for duplicate product names within a category, and supports category-based searching. These forms use ADO.NET commands and the project's database helper to communicate with SQL Server. From an OOP perspective, the work uses classes and methods to keep form behaviour organized, encapsulation through class-level fields and event handlers, and inheritance through Windows Forms classes derived from Form. I also participated in integrating my modules with the shared database connection, project configuration, testing and final GUI integration.

4.3 OOP Concepts Used

Encapsulation, class-based organization, event-driven methods, and inheritance through Windows Forms Form classes.

Primary classes/files: Form1.cs, frmMain.cs, frmCategory.cs, AddProduct.cs and their associated Designer/resource files.

5. Database Contribution

GoMartDB is implemented using Microsoft SQL Server. The database contains five principal tables: tblAdmin, tblSeller, tblCategory, tblProduct and tblBill.

tblAdmin	Administrator credentials and identity	AdminID
tblSeller	Seller records and credentials	SellerID
tblCategory	Product categories	CatID
tblProduct	Product catalogue and inventory quantity	ProdID; ProdCatID
tblBill	Billing records	Bill_ID

5.1 SQL Features

- Primary keys and identity columns.
- Product-to-category foreign-key relationship through ProdCatID.
- Stored procedures for CRUD and listing operations.
- Duplicate-product checking and category-based product search.
- Parameterized SqlCommand access in relevant application operations.

5.2 JOIN / Query Evidence

The supplied project documentation identifies product-listing procedures such as spGetAllProductList and spGetAllProductList_SearchByCat, which retrieve products together with category information. No specific GROUP BY query was documented, so none is claimed here.

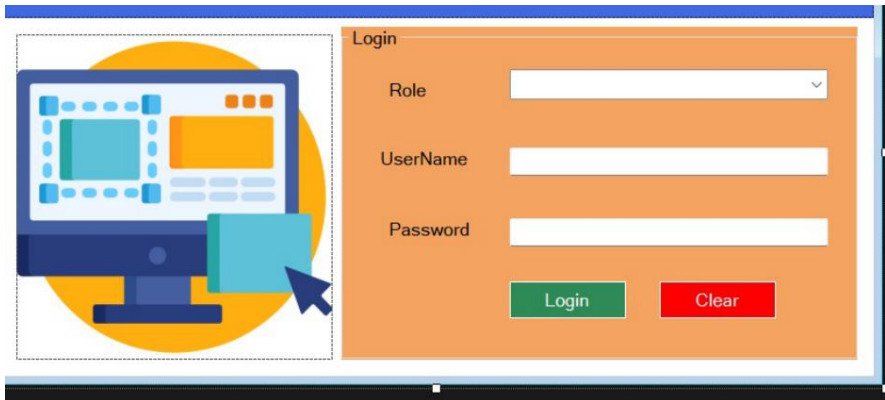
Shared database work includes the SQL setup script, stored procedures, database package and application-level database integration. Both members used the database resources as part of the final application.

6. GUI / Windows Forms Evidence

The application contains eight principal Windows Forms modules. The screenshots below are included directly in this report so the GUI evidence is visible without opening the repository.

Form1	Login and authentication
frmMain	Home/main menu and role-based navigation
frmCategory	Category management
AddProduct	Product management
frmAddNewSeller	Seller management
AddAdmin	Administrator management
SellingForm	Point-of-sale / billing
AboutBox1	About information

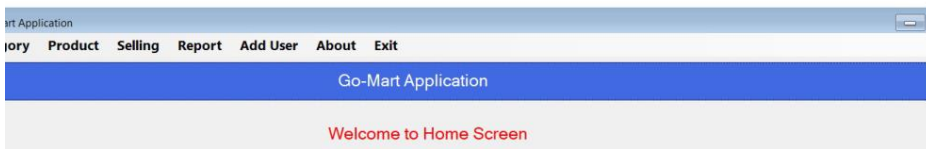
Login



The login screen features a blue header bar. On the left, there is a graphic of a computer monitor and keyboard with a blue arrow pointing to a login form. The form itself has an orange background and is titled 'Login'. It contains three input fields: 'Role' (a dropdown menu), 'UserName' (a text box), and 'Password' (a text box). Below these fields are two buttons: a green 'Login' button and a red 'Clear' button.

Figure 9.1 — Login screen with role selection

ne / Main Menu



The main menu screen has a blue header bar with the text 'Go-Mart Application'. Below the header is a white bar with a menu list: 'Home', 'Product', 'Selling', 'Report', 'Add User', 'About', and 'Exit'. The main content area has a light gray background and displays the text 'Welcome to Home Screen' in red.

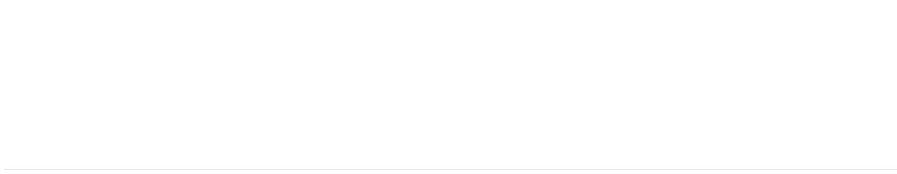
Role selection and credential entry.

Home / Main Menu



Figure 9.2 — Home screen with the main menu strip

.3 Category Management



Role-based navigation after login.

Category Management



The screenshot shows a web application window titled "Go-Mart Application". On the left, there is a form titled "Add Category". The form has a label "label4" above a text input field for "CategoryName:". Below that is a larger text area for "Category Description:". At the bottom of the form are three buttons: "Add" (green), "Update" (purple), and "Delete" (red). To the right of the form is a large, empty gray rectangular area, likely intended for displaying a list of categories.

Figure 9.3 — Adding and listing product categories

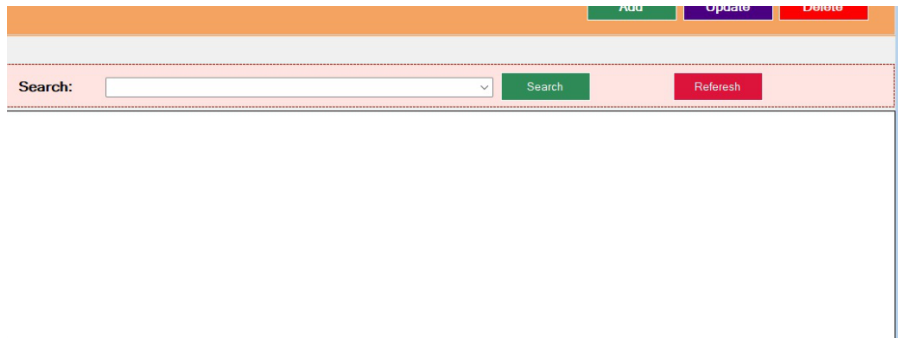
4 Product Management



The screenshot shows a web application window with a title bar that says "AddProduct". Below the title bar, there is a section titled "Manage Products" on an orange background. The rest of the window content is not visible.

Add, update and delete product categories.

Product Management



The screenshot displays the 'Product Management' interface. At the top, there is a navigation bar with three buttons: 'Add' (green), 'Update' (purple), and 'Delete' (red). Below this is a search section with a label 'Search:', a text input field, a dropdown arrow, a green 'Search' button, and a red 'Refresh' button. The main area below the search bar is a large, empty white space, likely intended for a list of products.

Figure 9.4 — Adding, searching, and listing products

5 Seller Management

Product entry, validation, search and category filtering.

Seller Management

Add New Seller
label4

Seller Name: Seller Age:

Phone Password

Figure 9.5 — Adding and listing sellers

6 Selling / Billing

SellingForm

Selling Form

Date: Jan 2023

Seller account management.

Selling / Billing

Quantity:

Add Order

Amount:

0.0

Search

Refresh

Sells List

Bill Number

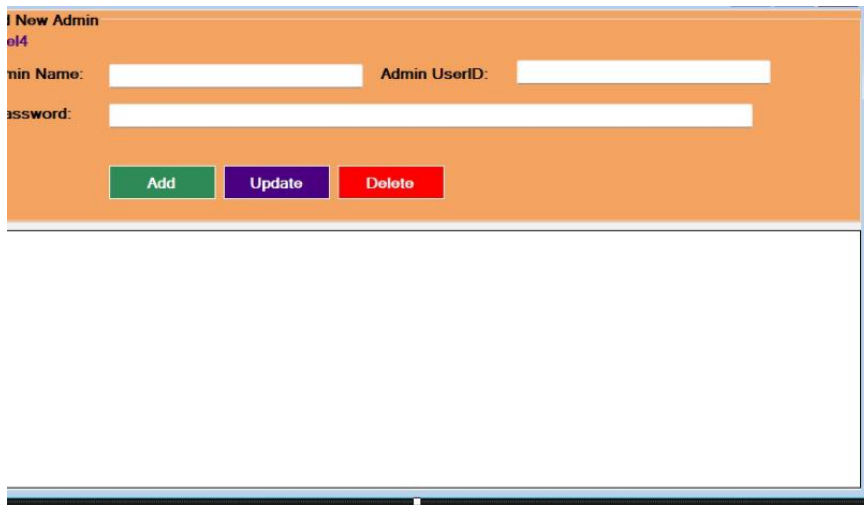
Add

Figure 9.6 — Point-of-sale billing screen with order list and bill history

.7 Admin Management

Point-of-sale order entry and bill history.

Admin Management



The screenshot shows a web form titled "Add New Admin" with a blue header bar. Below the header, there are three input fields: "Admin Name:", "Admin UserID:", and "Password:". At the bottom of the form, there are three buttons: "Add" (green), "Update" (purple), and "Delete" (red). The form is set against a light blue background.

Figure 9.7 — Adding and listing admins



Administrator account management.

About



dialog, populated from the assembly's title/version/description/company/copyright attributes

Application information dialog.

7. System Architecture & Diagrams

GoMart follows a simple layered structure: Windows Forms presentation layer → DBConnect/ADO.NET data-access layer → GoMartDB SQL Server database.

System Architecture

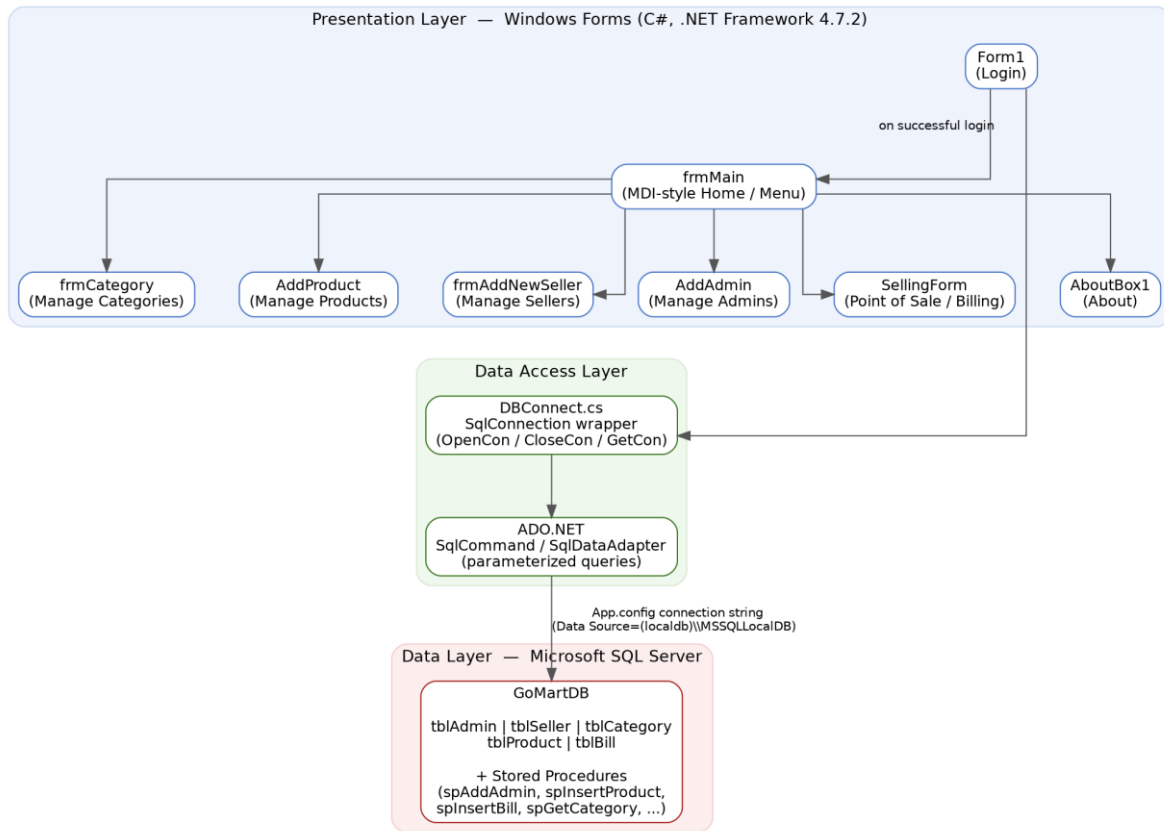


Figure 7.1 — GoMart system architecture

Login & Role-Based Access Flow

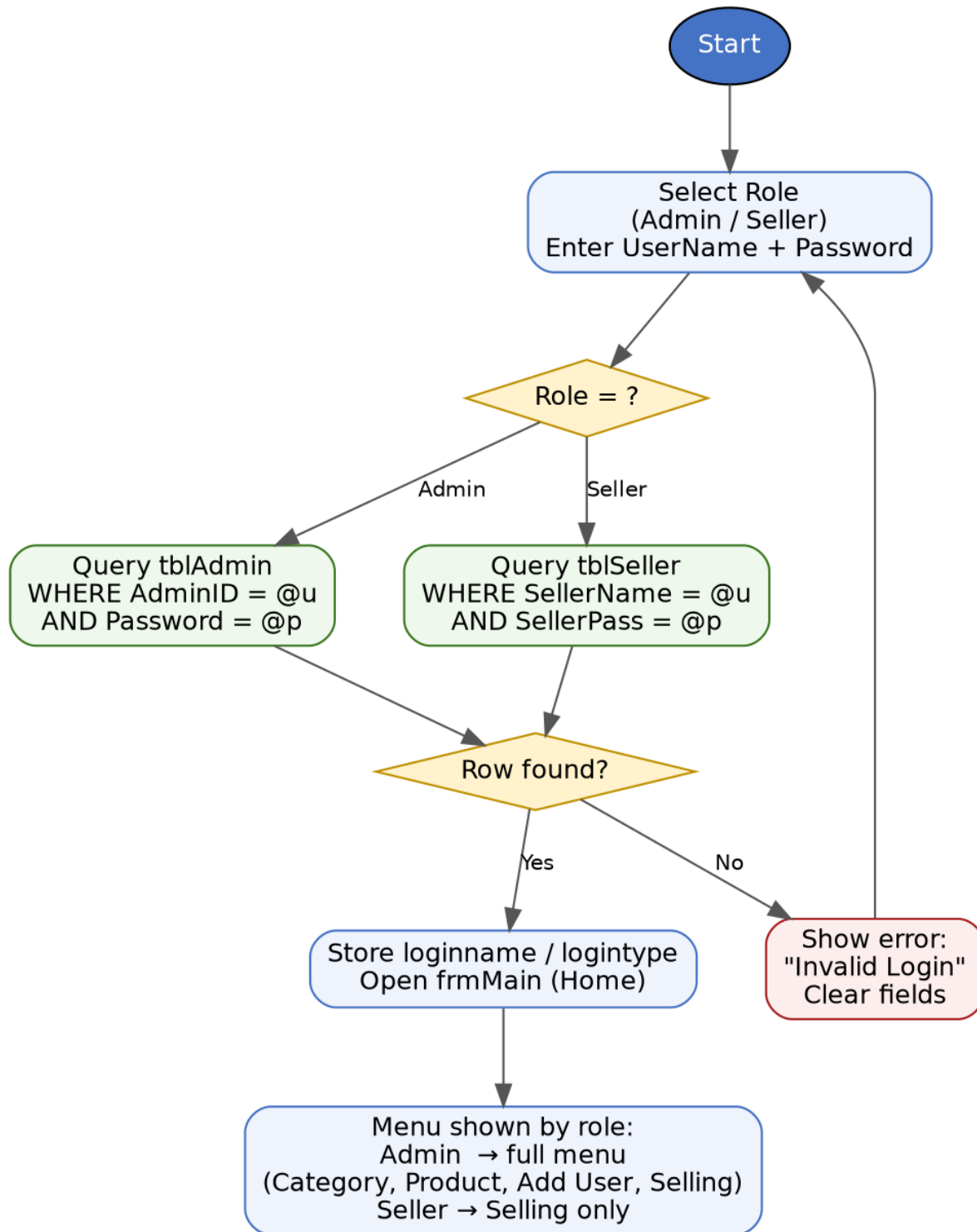


Figure 7.2 — Login and role-based access flow

Entity-Relationship Diagram

GoMart Database — Entity-Relationship Diagram

GoMartDB · Microsoft SQL Server · 5 tables

Legend

PK Primary key **U** Unique

FK Declared/used as foreign key

FK* App-level reference only —
no FOREIGN KEY constraint in schema

— solid = table has FK column

----- dashed = logical link, no FK column

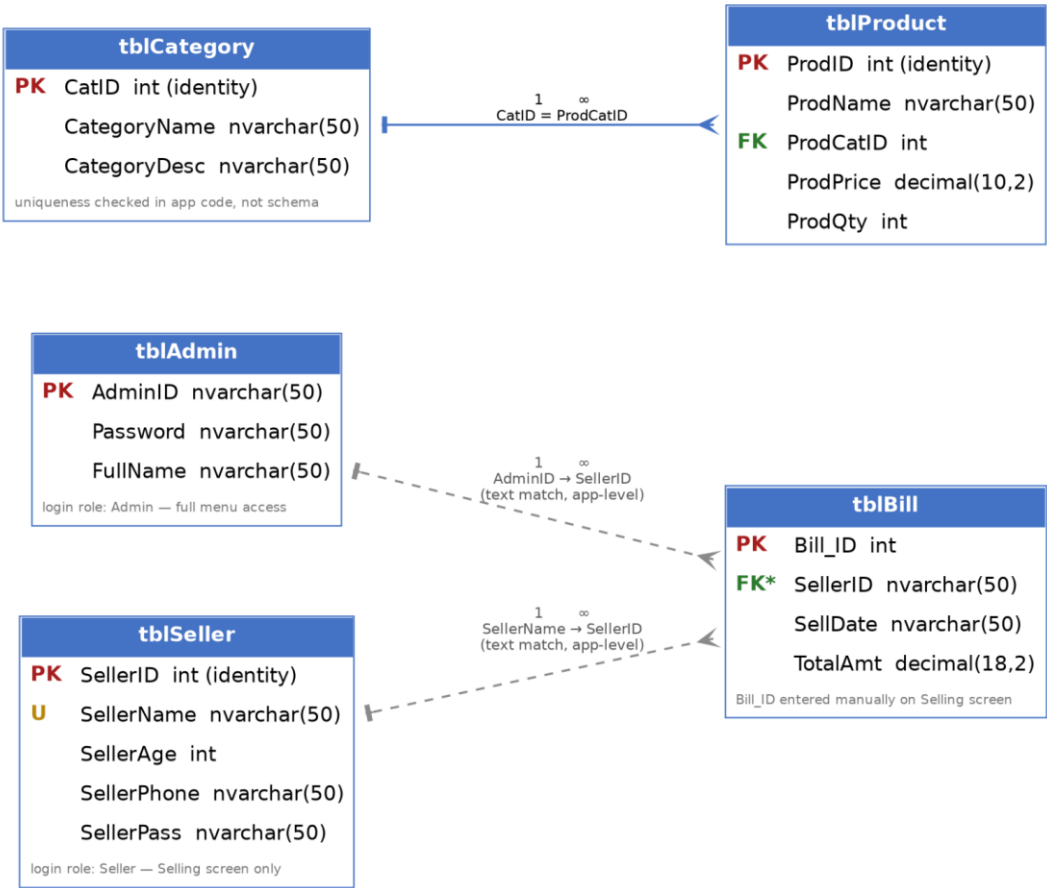


Figure 7.3 — GoMartDB entity-relationship diagram

8. Testing & Run Verification

Login	Valid/invalid credentials, empty-field validation and role-based navigation.
Category	Create, update, delete, list and duplicate-name rejection.
Product	Validation, duplicate checking, category filtering and CRUD.
Seller/Admin	Add, update, delete and duplicate checks.
Selling	Product selection, quantity/order total, bill saving and bill history.
Database	SQL connectivity and stored-procedure/database operations.

8.1 Run Verification

- Run Database/SQL_DB/GoMartDB_SQLServer_Setup.sql in SQL Server Management Studio or Azure Data Studio.
- Alternatively import Database/GoMartDatabase.bacpac.
- Open GoMartApplication.sln in Visual Studio.
- Confirm the SQL Server connection string in App.config.
- Build and run the application.
- The supplied documentation identifies a demo Admin account: Coder / 12345.

8.2 Known Limitations

- Passwords are stored as plain text in the current project.
- The login form currently uses an inline parameterized lookup rather than the existing login stored procedures.
- Stock quantity is not automatically reduced after a sale.
- Bill number and SellDate fields are more loosely typed than ideal.

8.3 Future Scope

- Hash and salt passwords.
- Move login authentication to stored procedures.
- Automatically decrement stock and add low-stock warnings.
- Improve bill numbering, date typing and relational constraints.
- Add printable receipts, barcode support, images and richer reports.
- Consider Entity Framework or Dapper for the data layer.

9. Academic Integrity & AI Disclosure

The contribution percentage and module distribution in this report are presented as the agreed 50/50 group split. GitHub commit evidence such as exact commit counts and dates should remain consistent with the actual repository history.

If AI tools were used for drafting, explanation, debugging or documentation, the final submission should disclose that assistance according to the course/faculty requirements. AI assistance does not replace the student's responsibility to understand and explain the submitted code.

10. Declaration

I, Maisha Jannat Monzur (ID 22-47624-2), declare that this individual report represents my assigned portion of the GoMart Application and that the stated 50% contribution reflects the group-agreed distribution of the project implementation.

Student	Maisha Jannat Monzur
Student ID	22-47624-2
Signature	Maisha Jannat Monzur
Date	14-09-2026

Appendix — Submission Checklist

- Public GitHub repository is enabled.
- README.md is present.
- Application/source package is uploaded.
- Database and SQL files are available.
- ER, architecture and login/role diagrams are uploaded.
- GUI screenshots are uploaded.
- This individual report is uploaded under Documentation/Report/.
- The group collaborator is added through GitHub repository settings.
- Actual GitHub commit history is preserved.
- Contribution percentage matches the group agreement.